Tourism web page guide

Please retrieve the file from https://github.com/Darylw3103/TourismSG.git. If request is needed please email me at: 2401117@sit.singaporetech.edu.sg and send me your email too.
To look at our firebase database, please email your firebase registered account to 2401117@sit.singaporetech.edu.sg to gain access. Thank you.

1. **System requirement**
    a. Node.js (v18+)
    b. npm install (comes with Node)
    c. Python 3.10+
    d. MySQL Server (8.0+)
    e. Firebase Project (Firestore + Storage)
    f. React + Vite
2. **Project Folder Overview**
    a. /Backend
        i. server.js
        ii. db.js
        iii. FirebaseAdmin.js
        iv. FirebaseImage.js
        v. Convert_foodplaces.js
        vi. .env
        vii. routes/
            1. attractions.js
            2. auth.js
            3. booking.js
            4. chat.js
            5. chatlogs.js
            6. concerts.js
            7. foodplaces.js
            8. hotels.js
            9. review.js
            10. userRoutes.js
    b. /ai_service
        i. Ai_service.py
        ii. Train_intent.py
        iii. models/intent_model
            1. Config.cfg
            2. Meta.json
            3. Tokenizer

c. /tourismbored
    i. /src
        1. Admin.jsx
        2. Attractions.jsx
        3. AuthContext.jsx
        4. BookingModal.jsx
        5. Chatwidget.jsx
        6. Concerts.jsx
        7. DetailPage.jsx
        8. firebaseConfig.js
        9. Foodplaces.jsx
        10. Header.jsx
        11. Hotels.jsx
        12. Landingpage.jsx
        13. Login.jsx
        14. Main.jsx
        15. Map.jsx
        16. Profile.jsx
        17. Reviewbox.jsx
        18. Signup.jsx
        19. Usersettings.jsx
        20. /dashboard
            a. activityPicker.jsx
            b. Bookinglist.jsx
            c. Dashboard.jsx
            d. itineraryCard.jsx
            e. ItineraryForm.jsx
            f. ItineraryPlanner.jsx
            g. Merlino.jsx
            h. MerlinoChatSideBar.jsx
            i. RecommendedList.jsx

3. **Program Setup (Node + Express)**
    a. Inside /backend and /frontend:
        i. Npm install (CMD)
    b. Editing .env file:
        i. Edit the .env file to your own MySQL password and user
    c. Start the program (backend & frontend)
        i. In one terminal do: cd tourism_backend/ npm start
        ii. Open second terminal do: cd ai_service/ py train_intent/ py ai_service.py
        iii. Open third terminal do: cd tourismbored/ npm start
4. **Web page Login detail**
    a. Admin:
        i. Email: admin@gmail.com
        ii. Pass: Admin123

b. User:
    i. Email: e@e.com
    ii. Pass: e
    iii. Alternative option: please sign up with your own desired email/password 😀🙄

5. AI Infomation

a. Intent Model (AI_Service.py - SpaCy)
    i. Trained a custom SpaCY text-classification model (intent_model/) using train_intent.py
    ii. The model predicts high-level intents:
        1. General_query
        2. Category_search
        3. General_attractions

    This forms the core intelligence that detects what the user wants.

b. NLP Extraction Beyond the Mode

    In addition to the trained model, the AI uses custom NLP logic to extract:
    i. Category
        1. E.g., “museum”, “concert”,”hotel”
    ii. Region
        1. Using a region list and phrase matcher
    iii. Cuisine
        1. E.g., “Japanese”, “Italian”
    iv. Hotel Type
        1. E.g., “budget hotel”, “luxury”, “3-star”

    This makes the bot more accurate than just a pure intent model

c. Chatbot -> SQL + NoSQL Hybrid Data Access
    i. The AI reads from SQL

    Depending on the user intent, the chatbot calls these backend endpoints:
        1. /api/attractions
        2. /api/hotels/search
        3. /api/foodplaces/search
        4. /api/concerts

        These endpoints pull actual data from MySQL:
        5. Attraction details
        6. Hotel info + price tiers
        7. Food places
        8. Upcoming concerts

    Strucutred facts always come from SQL, because SQL ensures consistency and avoids confllicting data

d. AI -> Firestore Writes

    Everytime the user chats, the backend saves the conversation into Firestore
    i. Firestore stores:
        1. Chatlogs

2. Past itinerary data
3. Nested activities
4. Timestamps
5. sessionIds

e. AI Itinerary Generator
    i. The Generator learns from the user
        1. Past itineraries from Firestore
            a. Previous destinations
            b. Categories visited
            c. Regions visited
            d. Used to infer preference patterns
    ii. Chat history from Firestore
        1. It reads all past user messages and extract
        2. Region keywords
        3. Category keywords
        4. Cuisine preferences
    iii. SQL Bookings
        1. It reads booking records from MySQL:
            a. Hotel bookings -> determines starting region
            b. Concert bookings -> places concerts on the correct date